

Sujet TP n° 7 correction :

Question 1 :

```
a=Coccinelle("femelle",10,2)
b=Coccinelle("femelle",10,2)
c=Coccinelle("male",10,2)
population= [a,b,c]
population_suivante, nb_proies=evolution(population,200)
for jour in range(1,5):
    population,nb_proies=evolution(population,nb_proies)
    print("Jour", jour, ":")
    print("Coccinelles :", len(population))
print("Pucerons :", nb_proies)
```

Ce code commence par créer trois objets de type Coccinelle avec des paramètres différents (leur sexe et deux valeurs numériques, probablement liées à leur énergie, âge ou capacités). Ces objets sont stockés dans une liste appelée population. Ensuite, la fonction evolution(population, 200) est appelée : elle simule une première évolution de la population de coccinelles en partant de 200 pucerons, et renvoie deux résultats, la nouvelle population de coccinelles et le nombre de pucerons restant. Puis une boucle for est exécutée pendant 4 jours : à chaque jour, la fonction evolution est rappelée avec la population actuelle et le nombre de pucerons du jour précédent pour mettre à jour le système. Après chaque mise à jour, le programme affiche le jour, le nombre de coccinelles présentes et le nombre de pucerons. Ainsi, le code sert à simuler de manière dynamique l'évolution d'un écosystème sur plusieurs jours.

Question 2 :

```
def simulation_simple(population, nb_proies):
    jour = 0
    while jour < 30 and len(population) > 0 and nb_proies > 0:
        population, nb_proies = evolution(population, nb_proies)
        jour += 1
    return (len(population), nb_proies, jour)
```

teste console :

```
a=Coccinelle("femelle",10,2)
b=Coccinelle("femelle",10,2)
c=Coccinelle("male",10,2)
population= [a,b,c]
resultat = simulation_simple(population, 1000)
print(resultat)
```

Question 3 :

```
def chasser(self, nb_proies, nb_coccinelles):
    """
    Simule la chasse d'une coccinelle et met à jour son niveau de nutrition.
    :param nb_proies: (int) Le nombre total de proies disponibles
    :param nb_coccinelles: (int) Le nombre total de coccinelles
    :cu: nb_proies et nb_coccinelles doivent être des entiers positifs
    :return: (int) Le nombre de proies restantes après la chasse
    :exemples:
```

```

>>> c = Coccinelle("femelle", 10, 2)
>>> c.chasser(100, 5)
<un entier <= 100>
>>> c = Coccinelle("male", 10, 1)
>>> c.chasser(0, 5)
0
>>> c = Coccinelle("femelle", 10, 2)
>>> c.chasser(50, 0)
50
>>> c = Coccinelle("male", 10, 2)
>>> c.chasser("100", 5)
Traceback (most recent call last):
...
AssertionError: nb_proies doit être un entier
>>> c = Coccinelle("male", 10, 2)
>>> c.chasser(-10, 5)
Traceback (most recent call last):
...
AssertionError: nb_proies doit être positif
>>> c = Coccinelle("male", 10, 2)
>>> c.chasser(50, -3)
Traceback (most recent call last):
...
AssertionError: nb_coccinelles doit être positif
"""
assert isinstance(nb_proies, int), "nb_proies doit être un entier"
assert isinstance(nb_coccinelles, int), "nb_coccinelles doit être un entier"
assert nb_proies >= 0, "nb_proies doit être positif"
assert nb_coccinelles >= 0, "nb_coccinelles doit être positif"
if nb_coccinelles == 0:
    return nb_proies
proies_par_cocci = nb_proies / nb_coccinelles
if proies_par_cocci > 20:
    consommation = random.randint(12, 20)
elif proies_par_cocci > 10:
    consommation = random.randint(8, 15)
else:
    consommation = random.randint(3, 8)
consommation = min(consommation, nb_proies)
if consommation >= 10:
    self.niv_nutrition += 1
else:
    self.niv_nutrition = max(0, self.niv_nutrition - 1)
return nb_proies - consommation

```

Question 4 :

```

def reproduction(self):
    """
    Une femelle avec un niveau de nutrition >= 2 engendre exactement
    deux descendants : un mâle et une femelle.
    """
    descendants = []
    if self.sexe == "femelle" and self.niv_nutrition >= 2 and self.age >= 20:
        descendants.append(Coccinelle("male", 0, 0))
        descendants.append(Coccinelle("femelle", 0, 0))
        self.niv_nutrition = 0
    return descendants

```

Cette méthode reproduction simule la reproduction d'une coccinelle. Elle crée d'abord une liste vide descendants qui servira à stocker les nouveaux individus. Ensuite, elle vérifie trois conditions pour qu'une reproduction ait lieu : la coccinelle doit être une femelle (`self.sexe == "femelle"`), avoir un niveau de nutrition suffisant (au moins 2), et être assez âgée (au moins 20). Si ces conditions sont réunies, elle donne naissance à deux coccinelles : un mâle et une femelle, toutes deux initialisées avec un âge et une nutrition à 0. Après la reproduction, la femelle perd toute sa nutrition (`self.niv_nutrition = 0`), ce qui simule le coût énergétique de la reproduction. Enfin, la méthode renvoie la liste descendants, qui contient les nouveaux individus créés.

```
def a_survecu(self):  
    """  
    Met à jour l'âge de la coccinelle et indique si elle est encore en vie.  
    """  
    self.age += 1  
    if self.age >= self.esperance_de_vie:  
        return False  
    if self.niv_nutrition == 0:  
        if random.random() < 1/3:  
            return False  
    return True
```

Cette méthode `a_survecu` sert à déterminer si une coccinelle reste en vie après une mise à jour de son état. D'abord, elle augmente son âge de 1 à chaque appel (`self.age += 1`), ce qui simule le passage du temps. Ensuite, elle vérifie si la coccinelle a atteint ou dépassé son espérance de vie (`self.esperance_de_vie`) : si c'est le cas, elle meurt et la fonction renvoie `False`. Sinon, elle vérifie son niveau de nutrition : si celui-ci est égal à 0, la coccinelle est en situation critique et a une chance sur trois de mourir, grâce à `random.random() < 1/3`. Si aucune de ces conditions n'entraîne sa mort, la coccinelle survit et la fonction renvoie `True`. Cette méthode modélise donc la survie d'un individu en fonction de son âge et de son état nutritionnel.